

CPSC 440/540 Machine Learning (January-April, 2022)

Assignment 4 (due Friday April 1st at midnight)

For this assignment you can work in groups of 1-2. However, please only hand in one assignment for the group. **It is possible that some questions on this assignment will be cancelled, depending on the pace of lectures.**

1. Name(s): Wira Azmoon Ahmad
2. Student ID(s): 55246318

1 Gaussians

1.1 Gaussian Self-Conjugacy

Consider n IID samples x^i distributed according to a Gaussian with mean μ and covariance $\sigma^2 I$,

$$x^i \sim \mathcal{N}(\mu, \sigma^2 I).$$

Assume that μ itself is distributed according to a Gaussian

$$\mu \sim \mathcal{N}(\mu_0, \Sigma_0),$$

with mean μ_0 and (positive-definite) covariance Σ_0 . In this setting, the posterior for μ also follows a Gaussian distribution.

Derive the form of the posterior distribution, $p(\mu \mid X, \sigma^2, \mu_0, \Sigma_0)$.

Hint: the posterior is a product of Gaussian densities.

Answer:

$$\begin{aligned}
& \Pr(\mu \mid X, \sigma^2, \mu_0, \Sigma_0) \\
& \propto \Pr(X \mid \mu, \sigma^2) \Pr(\mu \mid \mu_0, \Sigma_0) \\
& \propto \exp \left(-\frac{1}{2} \sum_i (x^i - \mu)^T (\sigma^2 I)^{-1} (x^i - \mu) \right) \exp \left(-\frac{1}{2} (\mu - \mu_0)^T \Sigma_0^{-1} (\mu - \mu_0) \right) \\
& = \exp \left(-\frac{1}{2} \left[\sigma^{-2} \sum_i (x^i - \mu)^T (x^i - \mu) + (\mu - \mu_0)^T \Sigma_0^{-1} (\mu - \mu_0) \right] \right) \\
& = \exp \left(-\frac{1}{2} \left[\sigma^{-2} \sum_i (x_i^T x_i - 2\mu^T x_i + \mu^T \mu) + \mu^T \Sigma_0^{-1} \mu - 2\mu^T \Sigma_0^{-1} \mu_0 + \mu_0^T \Sigma_0^{-1} \mu_0 \right] \right) \\
& = \exp \left(-\frac{1}{2} \left[(n\sigma^{-2} \mu^T \mu + \mu^T \Sigma_0^{-1} \mu) + \left(-2\sigma^{-2} \mu^T \sum_i x_i - 2\mu^T \Sigma_0^{-1} \mu_0 \right) + \left(\sigma^{-2} \sum_i x_i^T x_i + \mu_0^T \Sigma_0^{-1} \mu_0 \right) \right] \right) \\
& = \exp \left(-\frac{1}{2} \left[(\mu^T (n\sigma^{-2} I + \Sigma_0^{-1}) \mu) + \left(-2\mu^T (\sigma^{-2} \sum_i x^i + \Sigma_0^{-1} \mu_0) \right) + \left(\sigma^{-2} \sum_i x_i^T x_i + \mu_0^T \Sigma_0^{-1} \mu_0 \right) \right] \right) \\
& \propto \mathcal{N}(\mu_N, \Sigma_N) \quad (\text{product of Gaussians is Gaussian}) \\
& \propto \exp \left(-\frac{1}{2} [(\mu - \mu_N)^T \Sigma_N^{-1} (\mu - \mu_N)] \right) \\
& = \exp \left(-\frac{1}{2} [\mu^T \Sigma_N^{-1} \mu - 2\mu^T \Sigma_N^{-1} \mu_N + \mu_N^T \Sigma_N^{-1} \mu_N] \right)
\end{aligned} \tag{1}$$

Comparing coefficients,

$$\Pr(\mu \mid X, \sigma^2, \mu_0, \Sigma_0) = \mathcal{N}(\mu_N, \Sigma_N)$$

where

$$\begin{aligned}
\Sigma_N &= (n\sigma^{-2} I + \Sigma_0^{-1})^{-1} \\
\mu_N &= \Sigma_N \left(\sigma^{-2} \sum_i x^i + \Sigma_0^{-1} \mu_0 \right)
\end{aligned}$$

1.2 Generative Classifiers with Gaussian Assumption

Consider the 3-class classification dataset in this image: In this dataset, we have 2 features and each colour represents one of the classes. Note that the classes are highly-structured: the colours each roughly follow a Gaussian distribution plus some noisy samples.

Since we have an idea of what the features look like for each class, we might consider classifying inputs x using a *generative classifier*. In particular, we are going to use Bayes rule to write

$$p(y^i = c \mid x^i, \Theta) = \frac{p(x^i \mid y^i = c, \Theta) \cdot p(y^i = c \mid \Theta)}{p(x^i \mid \Theta)},$$

where Θ represents the parameters of our model. To classify a new example $\tilde{x}^i$, generative classifiers would use

$$\hat{y}^i = \arg \max_{y \in \{1, 2, \dots, k\}} p(\tilde{x}^i \mid y^i = c, \Theta) p(y^i = c \mid \Theta),$$

where in our case the total number of classes k is 3.¹ Modeling $p(y^i = c \mid \Theta)$ is easy: we can just use a k -state categorical distribution,

$$p(y^i = c \mid \Theta) = \theta_c,$$

where θ_c is a single parameter for class c . The maximum likelihood estimate of θ_c is given by n_c/n , the number of times we have $y^i = c$ (which we've called n_c) divided by the total number of data points n .

Modeling $p(x^i \mid y^i = c, \Theta)$ is the hard part: we need to know the *probability of seeing the feature vector x^i given that we are in class c* . This corresponds to solving a density estimation problem for each of the k possible classes. To make the density estimation problem tractable, we'll assume that the distribution of x^i given that $y^i = c$ is given by a $\mathcal{N}(\mu_c, \Sigma_c)$ Gaussian distribution for a class-specific μ_c and Σ_c ,

$$p(x^i \mid y^i = c, \Theta) = \frac{1}{(2\pi)^{\frac{d}{2}} |\Sigma_c|^{\frac{1}{2}}} \exp \left(-\frac{1}{2} (x^i - \mu_c)^T \Sigma_c^{-1} (x^i - \mu_c) \right).$$

Since we are distinguishing between the probability under k different Gaussians to make our classification, this is called *Gaussian discriminant analysis* (GDA). In the special case where we have a constant $\Sigma_c = \Sigma$ across all classes it is known as *linear discriminant analysis* (LDA) since it leads to a linear classifier between any two classes (while the region of space assigned to each class forms a convex polyhedron as in k -means clustering and softmax classification). Another common restriction on the Σ_c is that they are diagonal matrices, since this only requires $O(d)$ parameters instead of $O(d^2)$ (corresponding to assuming that the features are independent univariate Gaussians given the class label). Given a dataset $\mathcal{D} = \{(x^i, y^i)\}_{i=1}^n$, where $x^i \in \mathbb{R}^d$ and $y^i \in \{1, \dots, k\}$, the maximum likelihood estimate (MLE) for the μ_c and Σ_c in the GDA model is the solution to

$$\arg \max_{\mu_1, \mu_2, \dots, \mu_k, \Sigma_1, \Sigma_2, \dots, \Sigma_k} \prod_{i=1}^n p(x^i \mid y^i, \mu_{y^i}, \Sigma_{y^i}).$$

This means that the negative log-likelihood will be equal to

$$\begin{aligned} -\log p(X \mid y, \Theta) &= -\sum_{i=1}^n \log p(x^i \mid y^i, \mu_{y^i}, \Sigma_{y^i}) \\ &= \sum_{i=1}^n \frac{1}{2} (x^i - \mu_{y^i})^T \Sigma_{y^i}^{-1} (x^i - \mu_{y^i}) + \frac{1}{2} \sum_{i=1}^n \log |\Sigma_{y^i}| + \text{const.} \end{aligned}$$

In class we stated the MLE for this model under the assumption that we use full covariance matrices and that each class has its own covariance.

¹The denominator $p(\tilde{x}^i \mid \Theta)$ is irrelevant to the classification since it is the same for all y .

1. Derive the MLE for the GDA model under the assumption of *common diagonal covariance* matrices, $\Sigma_c = D$ (d parameters). (Each class will have its own mean μ_c .)

Answer:

$$\begin{aligned}\nabla_{\mu_c} - \log p(X | y, \Theta) &= 0 \\ \sum_{i \in y_c} (x^i - \mu_c)^T \Sigma_c^{-1} (-1) &= 0 \\ \sum_{i \in y_c} x^i - \sum_{i \in y_c} \mu_c &= \mathbf{0} \\ \mu_c &= \frac{\sum_{i \in y_c} x^i}{n_c}\end{aligned}$$

Let $D = \text{diag } \sigma^2$.

$$\begin{aligned}\nabla_{\sigma^{-2}} - \log p(X | y, \Theta) &= 0 \\ \nabla_{\sigma^{-2}} \left(\sum_{i=1}^n \frac{1}{2} (x^i - \mu_{y^i})^T D^{-1} (x^i - \mu_{y^i}) - \frac{1}{2} \sum_i \log |D^{-1}| + \text{const.} \right) &= 0 \\ \nabla_{\sigma^{-2}} \left(\sum_{i=1}^n \sum_{j=1}^d \frac{1}{2} (x_j^i - \mu_{y^i, j})^2 \sigma_j^{-2} - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^d \log \sigma_j^{-2} \right) &= 0 \\ \sum_{i=1}^n (\text{diag}(x^i - \mu_{y^i})) (x^i - \mu_{y^i}) - n \sigma^2 &= 0 \\ \sigma^2 &= \frac{1}{n} \sum_{i=1}^n (\text{diag}(x^i - \mu_{y^i})) (x^i - \mu_{y^i}) \\ \Sigma_c = D = \text{diag } \sigma^2\end{aligned}$$

2. Derive the MLE for the GDA model under the assumption of *individual scaled-identity* matrices, $\Sigma_c = \sigma_c^2 I$ (k parameters).

Answer:

$$\mu_c = \frac{\sum_{i \in y_c} x^i}{n_c}$$

Let $\Sigma_c^{-1} = \sigma_c^{-2} I$.

$$\begin{aligned}\nabla_{\sigma_c^{-2}} - \log p(X | y, \Theta) &= 0 \\ \nabla_{\sigma_c^{-2}} \left(\text{Tr} \sum_{i \in y_c} ((x^i - \mu_c)(x^i - \mu_c)^T \sigma_c^{-2} I) - n_c \log |\sigma_c^{-2} I| \right) &= 0 \\ \nabla_{\sigma_c^{-2}} \left(\sigma_c^{-2} \text{Tr} \left(\sum_{i \in y_c} (x^i - \mu_c)(x^i - \mu_c)^T \right) - n_c \log \sigma_c^{-2d} \right) &= 0 \\ \text{Tr} \left(\sum_{i \in y_c} (x^i - \mu_c)(x^i - \mu_c)^T \right) - d n_c \sigma_c^2 &= 0 \\ \sigma_c^2 &= \frac{1}{d n_c} \text{Tr} \left(\sum_{i \in y_c} (x^i - \mu_c)(x^i - \mu_c)^T \right)\end{aligned}$$

3. When you run *example_generative* it loads a variant of the dataset in the figure that has 12 features and 10 classes. This data has been split up into a training and test set, and the code fits a k -nearest neighbour classifier to the training set then reports the accuracy on the test data (around $\sim 63\%$ test error). The k -nearest neighbour model does poorly here since it doesn't take into account the Gaussian-like structure in feature space for each class label. Write a function *gda* that fits a GDA model to this dataset (using individual full covariance matrices). [Hand in the function and report the test set accuracy.](#)

Answer: **Test Error with GDA: 0.416**

using Statistics , LinearAlgebra

```
function logprob(xi,theta,mu,Sigma)
    tot = log(theta) - 0.5 * logdet(Sigma)
    tot += - 0.5 * (xi-mu)'*inv(Sigma)*(xi-mu)
    return tot
end

function gda_predict(Xhat,X,y)
    (n,d) = size(X)
    (t,d) = size(Xhat)
    k = maximum(y)
    theta = zeros(k)
    mu = zeros(k,d)
    Sigma = zeros(k,d,d)
    for c in 1:k
        idx = findall(==(c), y)
        theta[c] = length(idx) / n
        Xc = X[idx,:]
        mu[c,:] = mean(Xc, dims=1)
        Sigma[c,:,:] = cor(Xc)
    end
    yhat = zeros(t)
    for i in 1:t
        bestlogprob = -Inf
        for c in 1:k
            clogprob = logprob(Xhat[i,:],theta[c],mu[c,:],Sigma[c,:,:])
            if clogprob > bestlogprob
                yhat[i] = c
                bestlogprob = clogprob
            end
        end
    end
    return yhat
end

function gda(X,y)
    # Implementation of GDA
    predict(Xhat) = gda_predict(Xhat,X,y)
    return GenericModel(predict)
end
```

4. In this question we would like to replace the Gaussian distribution of the previous problem with the more robust multivariate-t distribution so that it isn't influenced as much by the noisy data. Unlike the previous case, we don't have a closed-form solution for the parameters. However, if you run *example_student* it generates random noisy data and fits a multivariate-t model. By using the *studentT* model, write a new function *tda* that implements a generative model that is based on the multivariate-t distribution instead of the Gaussian distribution. [Report the test accuracy with this model.](#)

Answer: **Test Error with TDA: 0.195**

```
include("studentT.jl")

function tda_predict(Xhat,X,y)
    (n,d) = size(X)
    (t,d) = size(Xhat)
    k = maximum(y)

    subModel = Array{DensityModel}(undef,k)
    theta = zeros(k)

    for c in 1:k
        idx = findall(==(c), y)
        theta[c] = length(idx) / n
        Xc = X[idx,:]
        subModel[c] = studentT(Xc)
    end

    # PDFs where i,j corresponds to xi,class
    p = zeros(t,k)
    for c in 1:k
        p[:,c] = subModel[c].pdf(Xhat)
    end
    p ./= theta'

    idx = argmax(p,dims=2)
    yhat = zeros(t)
    for i in 1:t
        yhat[i] = idx[i][2]
    end
    return yhat
end

function tda(X,y)
    predict(Xhat) = tda_predict(Xhat,X,y)
    return GenericModel(predict)
end
```

Hints: you may be able to substantially simplify the notation in the MLE derivations if you use the notation $\sum_{i \in y_c}$ to mean the sum over all values i where $y^i = c$. Similarly, you can use n_c to denote the number of cases where $y_i = c$, so that we have $\sum_{i \in y_c} 1 = n_c$. Note that the determinant of a diagonal matrix is the product of the diagonal entries, and the inverse of a diagonal matrix is a diagonal matrix with the reciprocals of the original matrix along the diagonal.

For the implementation you can use the result from class regarding the MLE of a general multivariate Gaussian. At test time for GDA, you may find it more numerically reasonable to compare log probabilities rather than probabilities of different classes, and you may find it helpful to use the *logdet* function to compute the log-determinant in a more numerically-stable way than taking the log of the determinant. (Also, don't forget to center at training and test time.)

For the last question, you may find it helpful to define an empty array that can be filled with k *DensityModel* objects using:

```
subModel = Array{DensityModel}(undef,k)
```

1.3 Empirical Bayes

Consider the model

$$y^i \sim \mathcal{N}(w^T z^i, \sigma^2), \quad w_j \sim \mathcal{N}(0, \lambda^{-1}),$$

where z^i is a length- k non-linear transformation of the features x^i (like a polynomial basis or RBFs). The posterior distribution in this model has the form

$$w \sim \mathcal{N}(w^+, \Theta^{-1}),$$

where the posterior precision and mean are given by

$$\Theta = \frac{1}{\sigma^2} Z^T Z + \lambda I,$$
$$w^+ = \frac{1}{\sigma^2} \Theta^{-1} Z^T y,$$

and Z contains the z^i vectors in the rows. The marginal likelihood in this model is given by

$$p(y \mid X, \sigma^2, \lambda) = \frac{(\lambda)^{k/2}}{(\sigma\sqrt{2\pi})^n |\Theta|^{1/2}} \exp\left(-\frac{1}{2\sigma^2} \|Zw^+ - y\|^2 - \frac{\lambda}{2} \|w^+\|^2\right).$$

As discussed in class, the marginal likelihood can be used to optimize hyper-parameters like σ , λ , and even the basis Z .

The demo *example_basis* loads a dataset and fits a degree-2 polynomial to it. Normally we would use a test set to choose the degree of the polynomial but here we'll use the marginal likelihood of the training set. Write a function, *leastSquaresEmpiricalBayes*, that uses the marginal likelihood to choose the degree of the polynomial as well as the parameters λ and σ (you can restrict your search for λ and σ to powers of 2). [Hand in your code and report the marginally most likely values of the degree, \$\sigma\$, and \$\lambda\$.](#)

Answer:

```
include("leastSquares.jl")

function leastSquaresEmpiricalBayes(X,y)
    (n,d) = size(X)
    bLL = -Inf
    blambda = 0
    bsigma = 0
    bp = 0

    for p in 1:10
        k = p + 1
        Z = polyBasis(X,p)
        lambda = 0.25
        for i in 1:10
            lambda *= 2
            sigma = 0.25
            for j in 1:10
                sigma *= 2
                Theta = (1/sigma^2)*Z'*Z + lambda * I
                w = (1/sigma^2)*inv(Theta)*Z'*y
                LL = (k/2) * log(lambda) - n * log(sigma) - 0.5 * logdet(Theta)
                LL += (-1/(2*sigma^2)) * norm(Z*w - y)^2
                LL += (-lambda/2) * norm(w)^2
```



```

        if LL > bLL
            bLL = LL
            blambda = lambda
            bsigma = sigma
            bp = k
            println(bLL, ", ", bp, ", ", blambda, ", ", bsigma)
        end
    end
end
end

return (bLL, bp, blambda, bsigma)
end

```

$$p = 5, \sigma = 0.5, \lambda = 128$$

2 Markov Models

2.1 Inference with Discrete States

The function *example_markov.jl* loads the initial state probabilities and transition probabilities for a Markov chain model,

$$p(x_1, x_2, \dots, x_d) = p(x_1) \prod_{j=2}^d p(x_j \mid x_{j-1}),$$

corresponding to the “grad student Markov chain” from class.

1. Write a function, *sampleAncestral*, that uses ancestral sampling to sample a sequence x from this Markov chain of length d . [Hand in this code and report the univariate marginal probabilities for time 50 using a Monte Carlo estimate based on 10000 samples.](#)

Hint: you can use *sampleDiscrete* in *misc.jl* to sample from a discrete probability mass function using the inverse transform method.

Answer:

```
include("misc.jl")

function sampleAncestral(p1, pt, d)
    x = zeros{Int, d}
    x[1] = sampleDiscrete(p1)
    for i in 2:d
        x[i] = sampleDiscrete(pt[x[i-1], :])
    end
    return x
end

function monteCarlo(p1, pt, d)
    p = zeros{Float64, length(p1)}
    for i in 1:10000
        x = sampleAncestral(p1, pt, d)
        p[x[d]] += 1
    end
    return p / sum(p)
end
```

$$\Pr(x_{50}) = [0.0127, 0.3918, 0.0154, 0.0056, 0.1061, 0.0823, 0.3861]$$

2. Write a function, *marginalCK*, that uses the CK equations to compute the exact univariate marginals up to a given time d . Hand in this code, report all exact univariate marginals at time 50, and report how this differs from the marginals in the previous question.

Answer:

```
function marginalCK(p1,pt,d)
    p = p1
    for i in 2:d
        p = pt' * p
    end
    return p
end

marginalCK(p1, pt, 50) = [0.013932459379462246, 0.3862968631380267, 0.017023638417965385,
0.005423766509870507, 0.10414317829675787, 0.0842973337907814, 0.3888827604671344]
The values are very close to those obtained from the Monte Carlo simulation.
```

3. What is the state c with highest marginal probability, $p(x_j = c)$, for each time j ?
Answer: For $j < 50$, this is state $c = 2$. From $j \geq 50$, this is state $c = 7$.
4. Write a function, *viterbiDecode*, that uses the Viterbi decoding algorithm for Markov chains to find the optimal decoding up to a time d . Hand in this code and report the optimal decoding of the Markov chain up to time 50 and up to 100.

Answer:

```
function viterbiDecode(p1,pt,d)
    k = length(p1)
    M = zeros(d, k)
    B = zeros(Int, d, k)
    x = zeros(Int, d)

    # Maximizing
    M[1,:] = p1
    for i in 2:d
        # each xi
        for j in 1:k
            # element wise
            p = pt[:, j] .* M[i-1,:]
            B[i, j] = argmax(p)
            M[i, j] = p[B[i, j]]
        end
    end

    # Backtracking
    x[d] = B[d, argmax(M[d, :])]
    for i in d-1:-1:1
        x[i] = B[x[i+1]]
    end

    return x
end
```

[illegible]

- Report all the univariate conditional probabilities at time 50 if the student starts in grad school, $p(x_{50} = c \mid x_1 = 3)$ for all c . Hint: you should be able to do this by changing the input to the CK equations.

Answer:

```
marginalCK(p13, pt, 50) = [0.00833562744038283, 0.23111584827032783, 0.01018545222724647,
0.008488559291948646, 0.1831155794085353, 0.16987617289442358, 0.38888276046713466]
```

- Report for all c the univariate conditional probabilities $p(x_5 = c \mid x_{10} = 6)$ (“where you were likely to be 5 years after graduation if you ended up in academia after 10 years”) obtained using a Monte Carlo estimate based on 10000 samples and rejection sampling. Also report the number of samples accepted among the 10000 samples.

Answer:

```
monteCarloRejection(p1, pt) = [0.004914004914004914, 0.029484029484029485, 0.19656019656019655,
0.0687960687960688, 0.16953316953316952, 0.5307125307125307, 0.0]
with 407 accepted samples.
```

- Give code implementing a dynamic programming approach to exactly compute $p(x_5 = c \mid x_{10} = 6)$, and report the exact values for all c .

Answer:

```
include("marginalCK.jl")
```

```
function forwardCondition(p1, pt, c, d=10, xi=6)
    k = length(p1)
```

```
    pc = marginalCK(p1, pt, c)
    pd = zeros(k)
    pd[xi] = 1
```

```
    for i in d:-1:c+1
        pd = pt * pd
    end
```

```
    condp = pc .* pd
```

```
    return condp / sum(condp)
```

```
end
```

```
forwardCondition(p1, pt, 5) = [0.0015535818661923315, 0.03392822926174592, 0.24774224920740526,
0.05807013200237585, 0.16133831091031867, 0.49736749675196207, 0.0]
```

- Why is $p(x_j = 7 \mid x_{10} = 6)$ equal to zero for all j less than 10?

Answer: It is impossible to come back to life after being deceased, as given by the probability distribution of the last row of the transition matrix.

Hint: for some of the questions you may find it helpful to use a k by d matrix M to represent a dynamic programming table

2.2 Markov Chain Monte Carlo

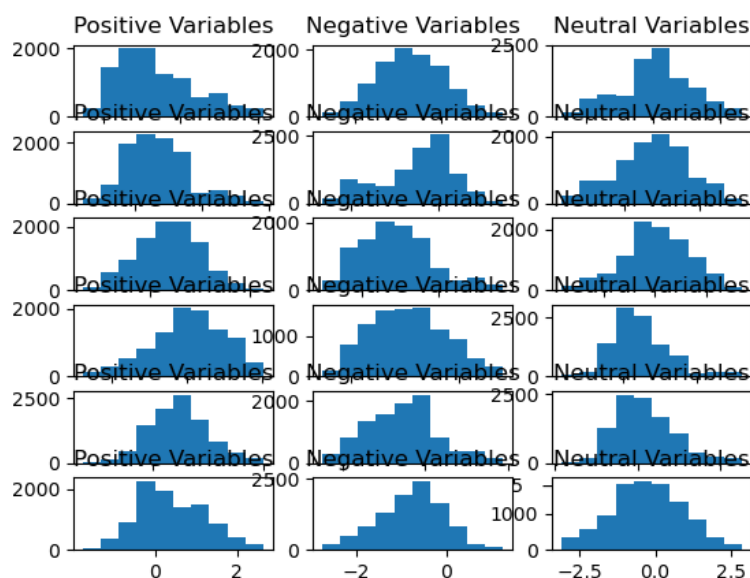
If you run *example_MH.jl*, it loads a set of images of ‘2’ and ‘3’ digits. It then runs the Metropolis MCMC algorithm to try to generate samples from the posterior over w , in a logistic regression model with a Gaussian prior. Once the samples are generated, it makes a histogram of the samples for several of the variables.²

1. Why would the samples coming from the Metropolis algorithm not give a good approximation to the posterior?

Answer: The variance is too large, causing new samples to be rejected and not contribute to the approximation.

2. Modify the proposal used by the demo to $\hat{w} \sim \mathcal{N}(w, (1/100)I)$ instead of $\hat{w} \sim \mathcal{N}(w, I)$. Hand in your code and the update histogram plot.

Answer: `wHat = w + randn(d,1) / 10`



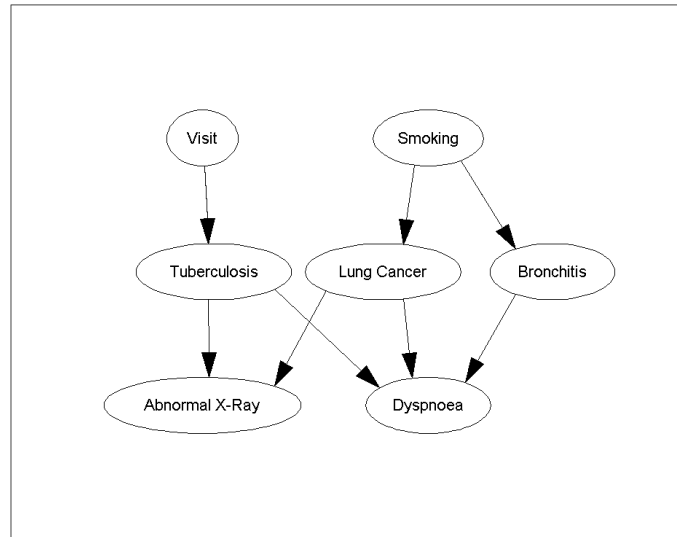
3. Modify the proposal to use $\hat{w} \sim \mathcal{N}(w, (1/10000)I)$. Do you think this performs better or worse than the previous choice? (Briefly explain.)

Answer: It performs worse as it does not provide a good representation of the sample space, exploring less due to the smaller variance.

²The “positive” variables are some of the positive weights when you fit an L2-regularized logistic regression model to the this data. The “negative” variables are some of the negative regression weights in that model, and the “neutral” ones are set to 0 in that model.

2.3 D-Separation

Consider the DAG model below, for distinguishing between different causes of shortness-of-breath (dyspnoea) and the causes of an abnormal lung x-ray, while modelling potential causes of these diseases too (whether the person is a smoker or had a ‘visit’ to a country with a high degree to tuberculosis).



We'll assume that the distribution over the variables is “faithful” to the graph, meaning that variables are conditionally independent if and only if the graph structure implies their independence. Under this assumption, [say why each of the following conditional independence statements is true or false \(provide a very-brief justification\)](#):

1. $(\text{Smoking}) \perp (\text{Bronchitis})$.
2. $(\text{Smoking}) \perp (\text{Dyspnoea})$.
3. $(\text{Tuberculosis}) \perp (\text{Lung Cancer})$.
4. $(\text{Abnormal X-Ray}) \perp (\text{Dyspnoea})$.
5. $(\text{Abnormal X-Ray}) \perp (\text{Visit}) \mid (\text{Tuberculosis})$.
6. $(\text{Bronchitis}) \perp (\text{Lung Cancer}) \mid (\text{Smoking})$.
7. $(\text{Tuberculosis}) \perp (\text{Lung Cancer}) \mid (\text{Dyspnoea})$.
8. $(\text{Visit}, \text{Smoking}) \perp (\text{Abnormal X-Ray}, \text{Dyspnoea}) \mid (\text{Tuberculosis}, \text{Lung Cancer}, \text{Bronchitis})$
9. $(\text{Smoking}) \perp (\text{Visit}) \mid (\text{Dyspnoea})$.
10. $(\text{Tuberculosis}) \perp (\text{Bronchitis}) \mid (\text{Abnormal X-Ray})$.

Answer:

1. False. Smoking directly affects Bronchitis.
2. False. Smoking affects Dyspnoea via Lung Cancer and Bronchitis.
3. True. They have no common parent and unobserved children.
4. False. They have common unobserved parents Tuberculosis, Lung Cancer.
5. True. Dependency of Visit with Abnormal X-Ray is broken by Tuberculosis.
6. True. Their single common observed parent Smoking breaks their dependency.
7. False. Knowing Dyspnoea means knowing Tuberculosis can give information about Lung Cancer.
8. True. Dependency of Visit, Smoking with Abnormal X-Ray, Dyspnoea is broken by Tuberculosis, Lung Cancer, Bronchitis.
9. False. Observing Dyspnoea, the grandchild of Visit and Smoking, creates dependence between Visit and Smoking.
10. False. Knowing Abnormal X-Ray would have Tuberculosis dependent with Lung Cancer, which is then dependent on Bronchitis without knowing Smoking.

3 Mixture Models

Cancelled.

4 Very-Short Answer Questions

1. What is the relationship between using a product of Gaussians and using a multivariate Gaussian?

Answer: Product of Gaussians is a special case of the multivariate Gaussian where each variable follows an independent Gaussian with their own mean and variance.

2. How does the affine property allow us to sample from multivariate Gaussians?

Answer: We can sample from a simpler Gaussian such as $\mathcal{N}(0, I)$ and transform the sample so it comes from the desired Gaussian using the affine property.

3. With a Gaussian likelihood, a Gaussian prior for its mean, and a fixed covariance, how do we know that the posterior predictive will also be a Gaussian?

Answer: The Gaussian prior is the conjugate prior for the Gaussian, so we know the posterior and posterior predictive are also Gaussian.

4. How do the sparsity patterns of the covariance and precision matrix in a multivariate Gaussian relate to independence and conditional independence in the models?

Answer: Sparsity in the covariance matrix would imply independent variables, while sparsity in the precision matrix implies conditional independence, implied from the graph structure that can be drawn from it.

5. In linear discriminant analysis, why might we not assign a point to its closest mean?

Answer: The class proportions π_c might not be equal, and thus the most likely classification is not the one with the closest mean.

6. The maximum of the posterior predictive in Bayesian linear regression gives the same prediction as if we used the MAP estimate. What is a reason we might use Bayesian linear regression anyways?

Answer: We may want the variance of predictions, unavailable from just the MAP estimate.

7. What is the key difference between Monte Carlo approximations and variational approximations?

Answer: Monte Carlo uses the empirical distribution from sampling, while variational tries to approximate the distribution with a "closest" distribution from a tractable family, such as Gaussians.

8. What is a key advantage of "end-to-end" training of computer vision system?

Answer: It is much easier to write optimization code for the entire pipeline instead of separate components.

9. What are your 3 favourite properties of the exponential family?

Answer:

- (a) They are guaranteed to have conjugate priors.
- (b) The sufficient statistics of the data $s(X)$ can be drawn and contains everything needed for learning.
- (c) They provide simple/intuitive closed form solutions for the MLE.

10. What is the difference between computing marginals and computing the stationary distribution of a Markov chain.

Answer: The marginals are distributions at any point of time, while the stationary distribution is one that does not change over time. Under certain conditions, the marginals converge to a stationary distribution as time goes to ∞ .

11. What is the cost of generating a sample from a Markov chain of length d with k possible states for each time? What is the cost of decoding?

Answer:

Sampling:

$$O(dk)$$

$O(k)$ for each d times.

Decoding:

$$O(dk^2)$$

$O(k^2)$ computing all $M_j(x_j)$ for a fixed j, d times.

12. In what setting is it unnecessary to include the q function in the Metropolis-Hastings acceptance probability?

Answer: If q is symmetric, then $q(x | \hat{x}) = q(\hat{x} | x)$ and their ratio can be ignored.

13. What is “explaining away”?

Answer: Knowing one parent can make other parents more or less likely.

14. If two variables are not d-separated, are they necessarily dependent? If two variables are d-separated, are they necessarily independent?

Answer: No. Yes. (Respectively)

15. Describe how we could use ancestral sampling to sample from the joint density over x and y defined by a Gaussian discriminant analysis model.

Answer: First sample y from $\Pr(y)$, then sample x from $\Pr(x | y)$.

GPU Access

We applied for UBC to fund some GPUs (RTX 3070 8GB) and a lot of RAM (192 GB) for use by students for their coursework. This is the first semester that these resources will be used, so there may be a learning curve to use these (from your side and our side). If you would like to use these for your course project, please make a private post on Piazza with the following information:

1. Names of people on the project.
2. Estimated number of GPU/CPU hours.
3. Roughly how much memory your jobs will need.
4. Roughly how much disk usage you will need.
5. Are there datasets you plan to use that other groups might also be using?